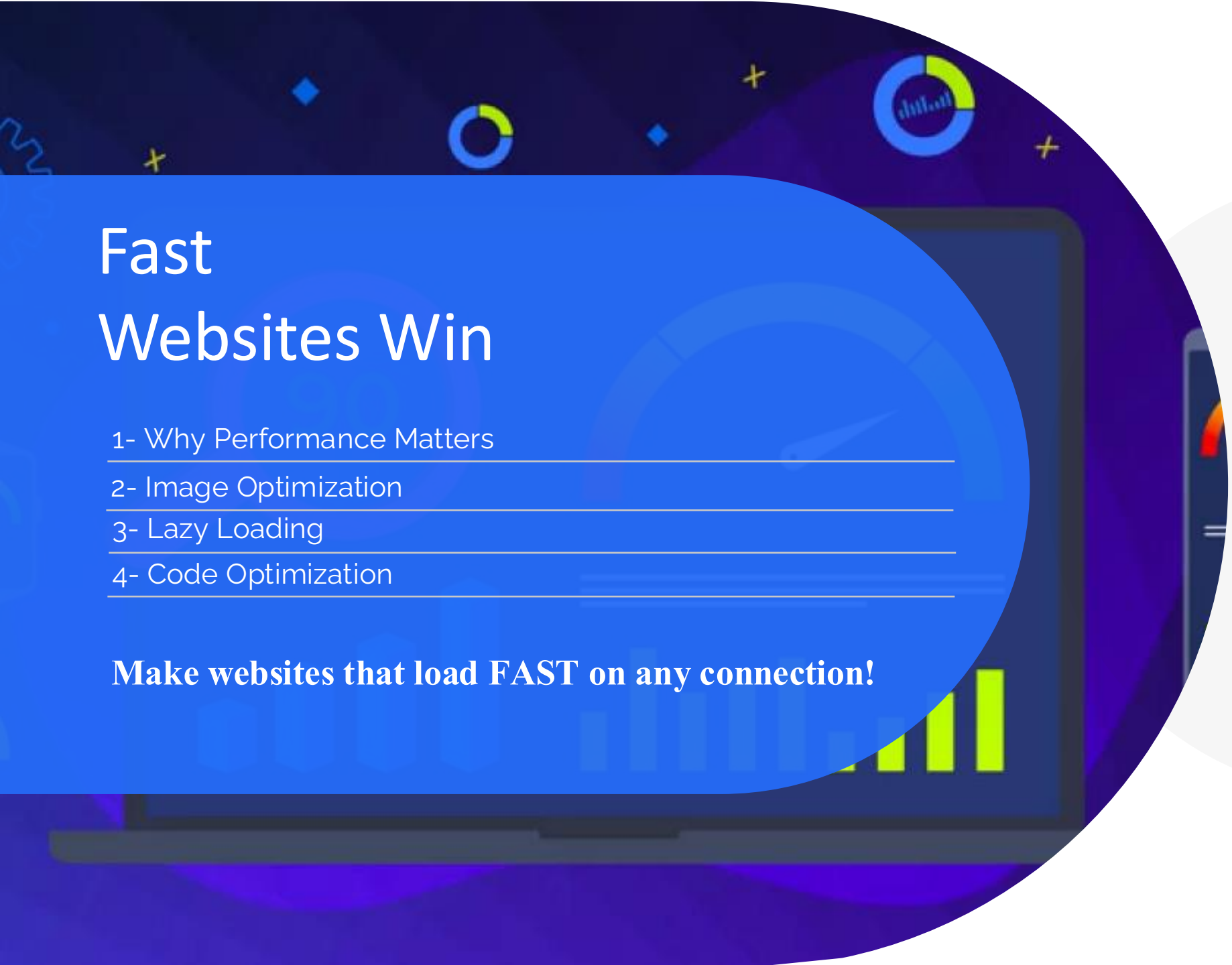


Fast Websites Win

1- Why Performance Matters

2- Image Optimization

3- Lazy Loading

4- Code Optimization

Make websites that load FAST on any connection!

SCROLL

Quick Recap

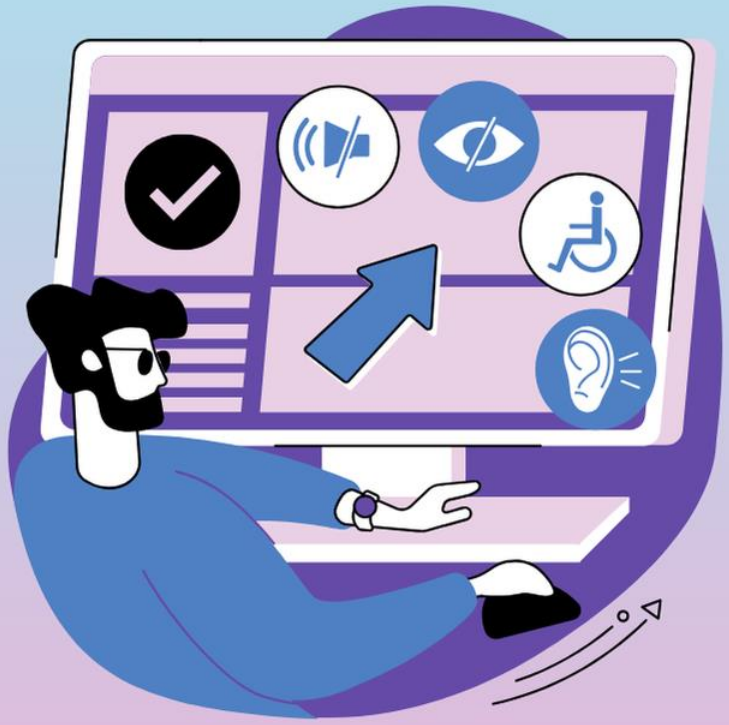
Last Week: Inclusive Design Practices

Cognitive & Inclusive Design Topics:

- Cognitive disabilities (ADHD, dyslexia, autism)
- Cognitive load & information overload
- Readable content and plain language
- Neurodiversity-aware design
- Internationalization (i18n)
- Cultural and language differences

Inclusive Design Strategies:

- Simple, clear language
- Consistent layouts and patterns
- Chunking content for scanning
- Strong visual hierarchy
- Reducing visual clutter
- Respecting motion sensitivity



What You Focused On:

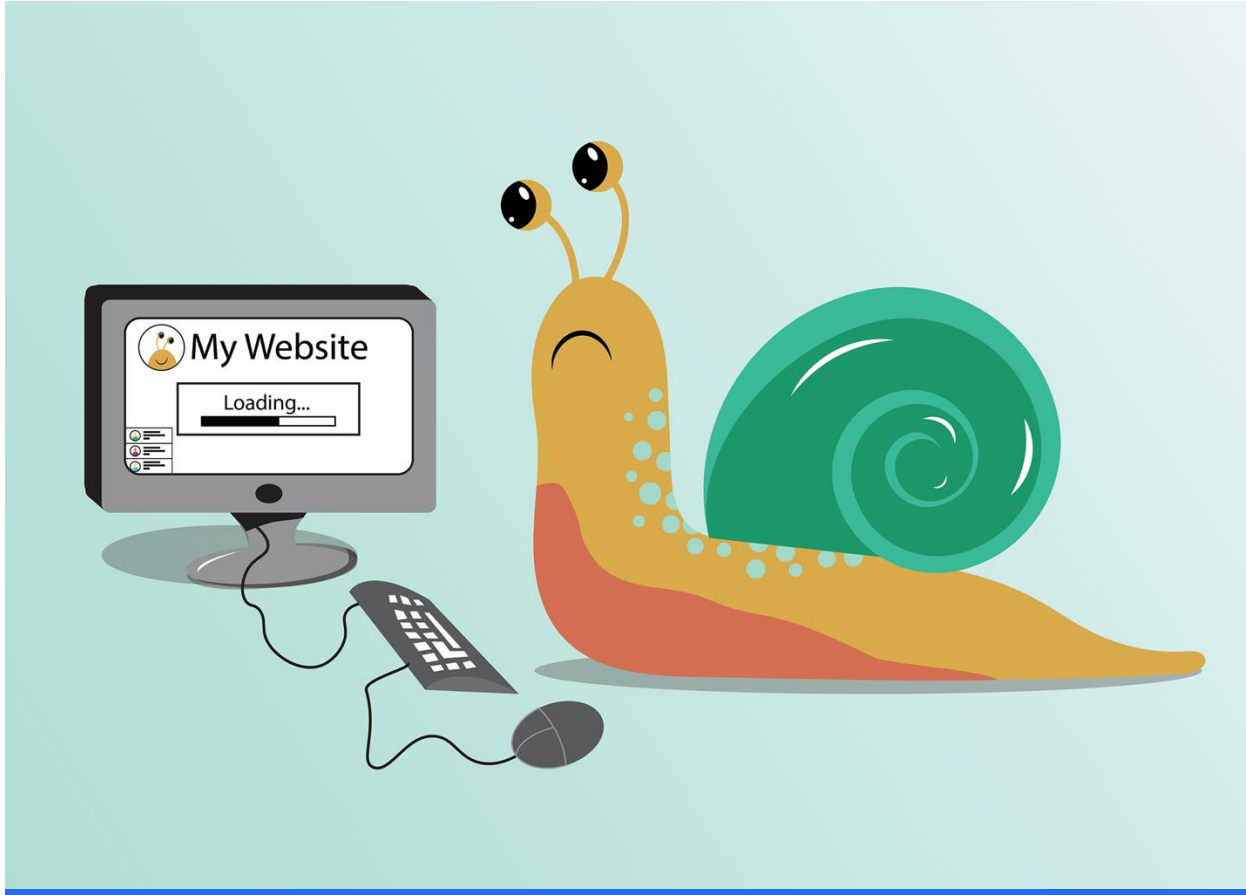
- Designing for how users **think**, not just how they see
- Reducing friction and mental effort
- Supporting global and neurodiverse users
- Testing designs with real users in mind

Fast vs Slow - The Experience Gap

The Performance Problem

Website A: Loads in 1 second
- User is happy
- User completes task
- User returns later

Website B: Loads in 5+ seconds
- User is frustrated
- User hits back button
- User finds competitor



Real Talk: Which one would YOU actually wait for?

Users Don't Wait - The Data

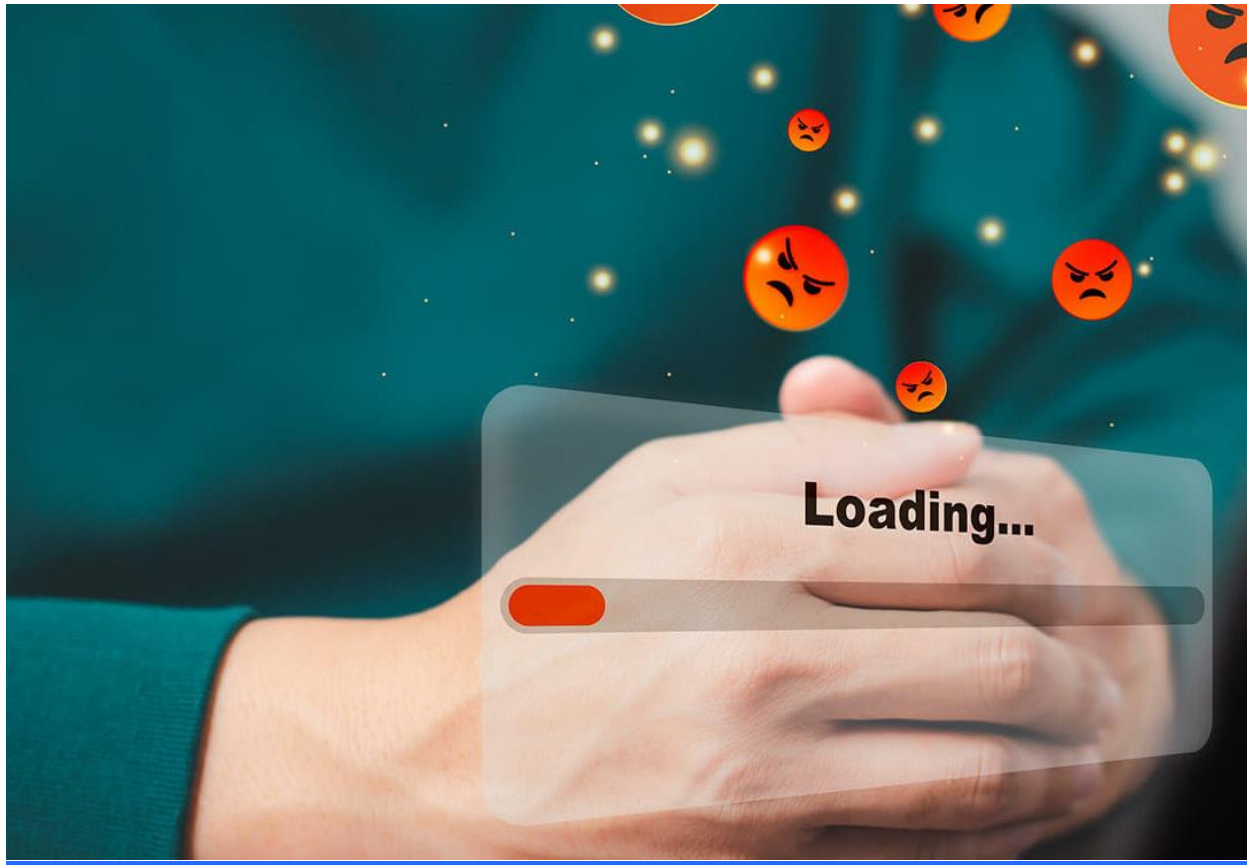
The Reality of User Behavior:

User Expectations: < 2 seconds
- That's their expectation for ANY website
- Doesn't matter if it's Amazon or your student project

Mobile Abandonment: 53% leave after 3 seconds
- More than half your visitors - gone
- They don't give you a second chance

Conversion Impact: Every 1 second delay = 7% fewer conversions
- Sales, signups, form submissions - all drop
- Stack up: 5 second delay = 35% loss!

Bottom Line: Slow sites = Lost users = Lost opportunities



The good news? You can control this!

Connection Reality in 2026

Reality Check: 5G Exists, But...

Yes, 5G is here! But connection quality varies:
Common Slow Connection Scenarios:

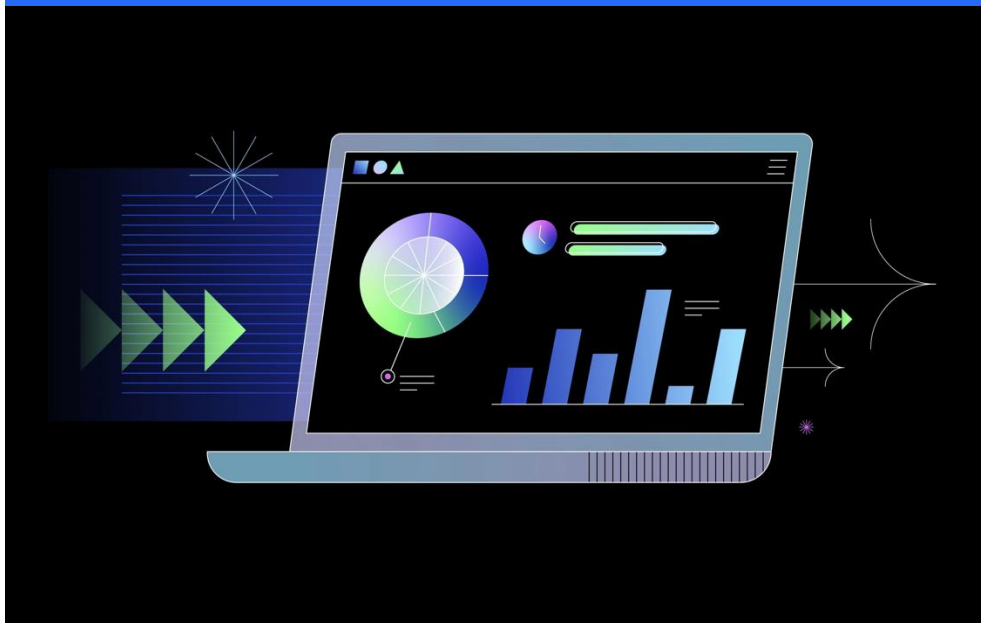
- Inside buildings (basements, offices, thick walls)
- Crowded events (concerts, sports, conferences)
- Rural and suburban areas between towers
- Traveling (airports, trains, highways)
- International roaming
- Network congestion during peak hours
- Older devices still on 4G/LTE



The Point: Even in 2025, connection speeds fluctuate constantly. Your site needs to work in ALL scenarios, not just perfect conditions.

Why Performance Matters

As designers and communicators, we have an ethical responsibility to make our work accessible to everyone, not just people with the latest devices and fastest internet.



Content: Four Critical Reasons:

1. User Experience

- Fast feels better and more professional
- Lower frustration, higher satisfaction
- Users return and recommend your site

2. Search Engine Rankings (SEO)

- Google uses speed as a ranking factor
- Fast sites rank higher in search results
- More visibility, more visitors

3. Business Impact

- Faster sites = more conversions
- Every second counts for revenue
- Professional credibility matters

4. Accessibility and Inclusion

- Fast sites work for more people
- Inclusive of different devices and connections
- Ethical responsibility to all users

Business Impact - Real Numbers

Real Numbers from Real Companies:



Amazon

- 100ms delay = 1% revenue loss
- At their scale: millions of dollars
- Every tenth of a second matters



Google

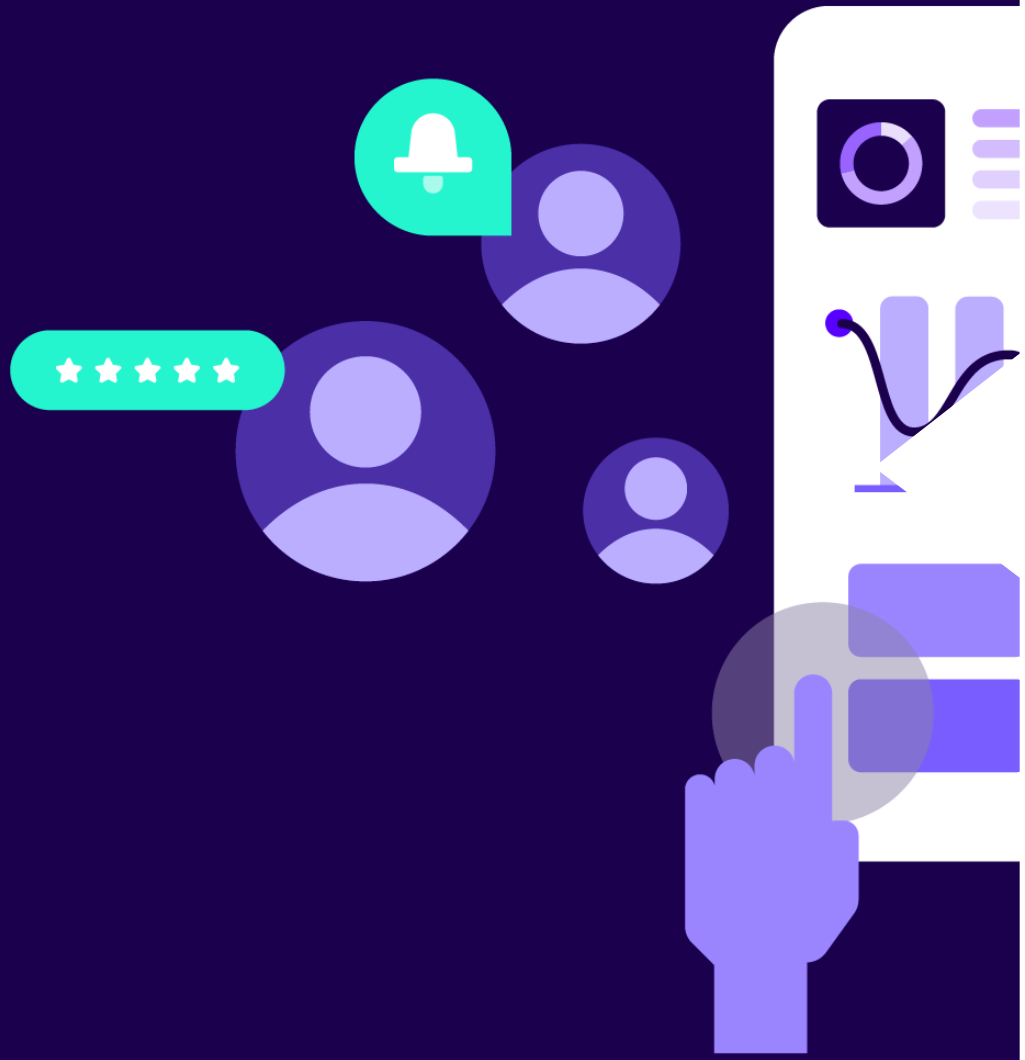
- 500ms delay = 20% traffic drop
- Half a second = one-fifth of visitors gone
- Led to massive optimization efforts



Pinterest

- 40% performance improvement = 15% more signups
- Faster site = more engaged users
- Directly measurable business growth

The Pattern: Milliseconds = Money. Speed = Success.



Performance IS Accessibility

Content: Slow Sites Exclude Real People:

Who Gets Left Behind:

- Users on older devices (can't afford latest phones)
- Users on slow connections (rural areas, congestion)
- Users with limited data plans (watching every MB)
- Users in developing countries (infrastructure limits)
- Low-income users (basic phones and plans)
- Users with cognitive disabilities (slow sites add confusion)

Fast Sites = Inclusive Sites

Performance is not optional. It's an accessibility requirement.

Core Web Vitals

Google's Three Key Performance Metrics:

01.

LCP - Largest Contentful Paint

- When main content becomes visible
- Usually your hero image or main headline
- Goal: Under 2.5 seconds
- Measures: "When can users see the important stuff?"

02.

FID - First Input Delay

- How quickly page responds to first click
- Time between click and browser response
- Goal: Under 100 milliseconds
- Measures: "Does the page feel responsive?"

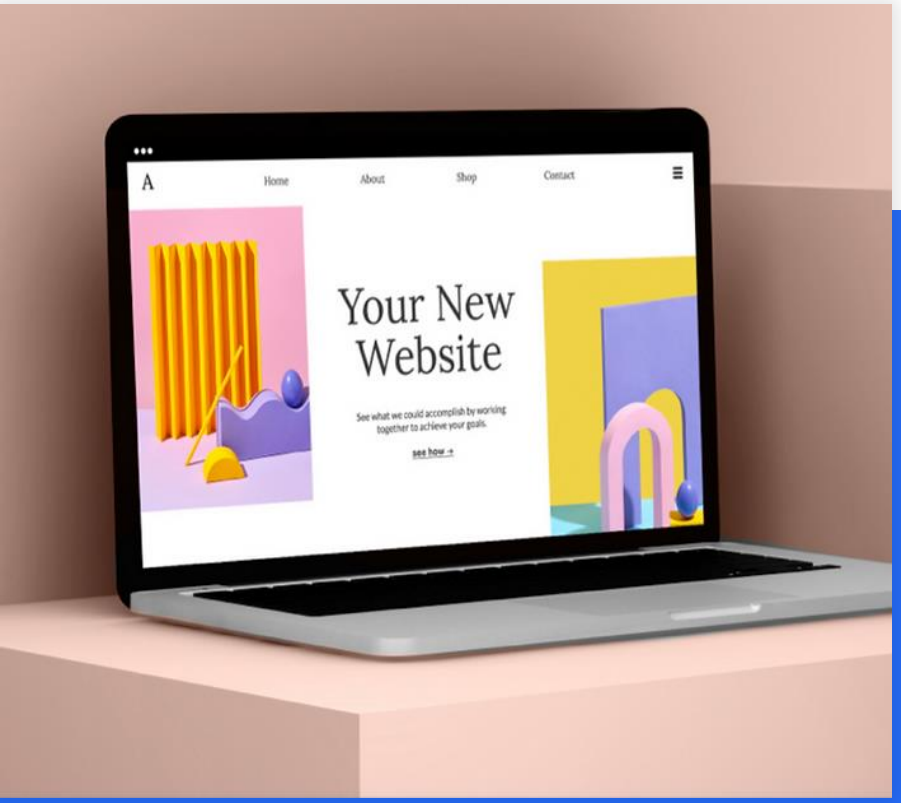
03.

CLS - Cumulative Layout Shift

- How much content jumps around while loading
- Prevents annoying "button moved as I clicked it"
- Goal: Under 0.1
- Measures: "Is the page stable or jumpy?"

You don't need to memorize these. Just know: Google cares deeply about speed.

The Number One Performance Killer



Focus your optimization efforts on images first for maximum impact.

Images Account for 50% of Page Weight

Typical Webpage Breakdown:

- **Images: 1 MB (50%)**
- JavaScript: 400 KB (20%)
- CSS: 100 KB (5%)
- HTML: 50 KB (2.5%)
- Fonts: 200 KB (10%)
- Everything else: 250 KB (12.5%)
- **Total: 2 MB**

The Good News: Biggest problem = Easiest to fix!

The Image Problem - A Real Example



Result: 23x smaller file, virtually identical visual quality

Real Example: Hero Image Optimization

Before Optimization:

- File size: 4.2 MB
- Dimensions: 3000 x 2000 pixels
- Format: JPG (from camera)
- Load time on 3G: 30+ seconds
- Load time on 5G: 3-5 seconds

After Optimization:

- File size: 180 KB
- Dimensions: 1200 x 800 pixels
- Format: WebP
- Load time on 3G: 2 seconds
- Load time on 5G: Under 1 second

Right-Sizing Images

Serve Images at Appropriate Sizes

Wrong Approach:

```
<!-- Serving 3000x2000px, displaying at 300x200px -->
<img src="huge-photo.jpg" width="300">
```

- Browser downloads 3000px wide image
- CSS shrinks it to 300px for display
- User wastes bandwidth on pixels they never see
- Page loads slowly for no reason

Right Approach:

```
<!-- Serve multiple sizes, browser picks best -->
<img src="photo-300.jpg"
     srcset="photo-300.jpg 300w,
             photo-600.jpg 600w,
             photo-1200.jpg 1200w"
     sizes="300px"
     alt="Description">
```

- Browser downloads only what it needs
- Mobile gets 300px, desktop gets 1200px
- Everyone wins

Image Compression Explained

Understanding Lossless vs Lossy Compression

Lossless Compression (PNG):

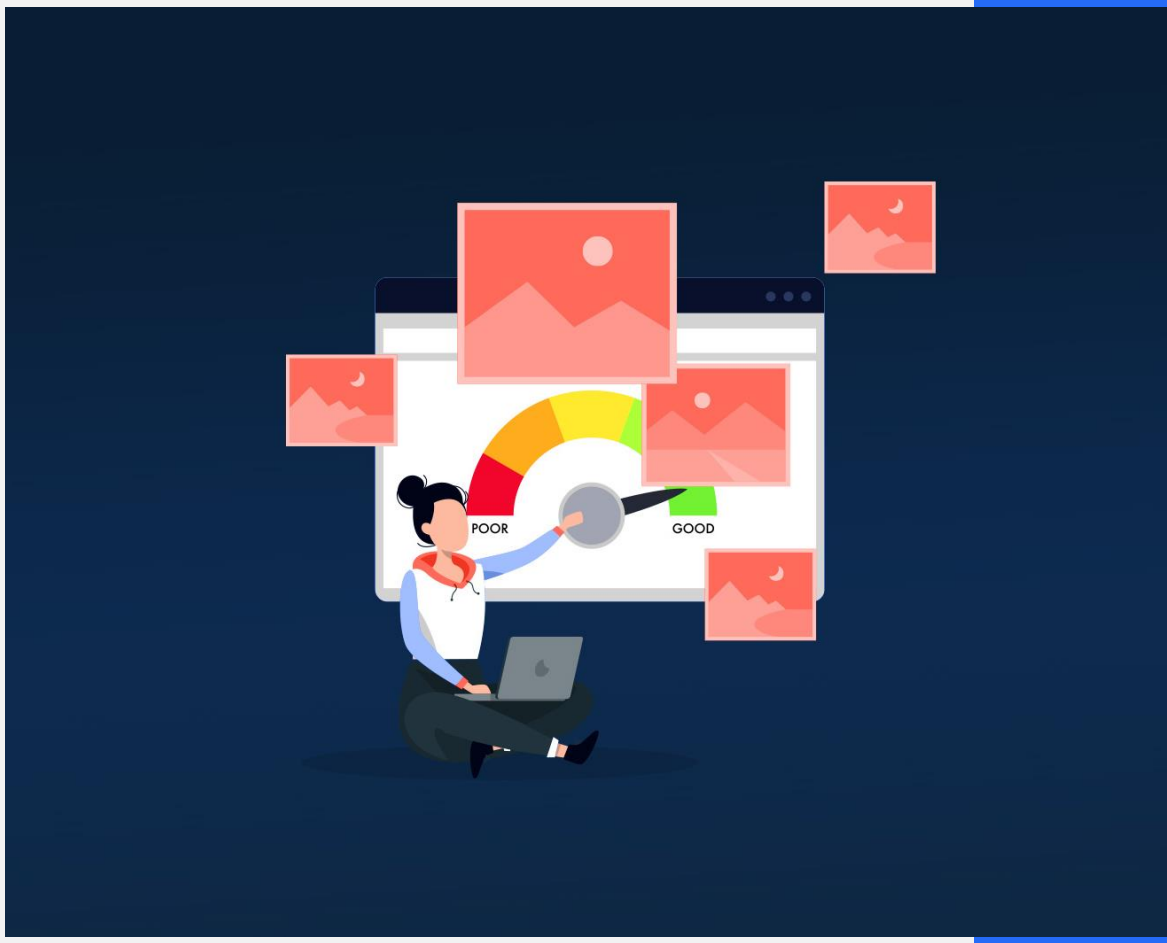
- Zero quality loss
- Every pixel preserved exactly
- Larger file sizes
- **Best for:** Logos, screenshots, text images, graphics with sharp edges
- **Example:** 500 KB for logo

Lossy Compression (JPG/WebP):

- Slight quality loss (invisible to human eye)
- Much smaller file sizes
- **Best for:** Photos, hero images, portraits, natural scenes
- **Example:** Same photo at 50 KB instead of 500 KB

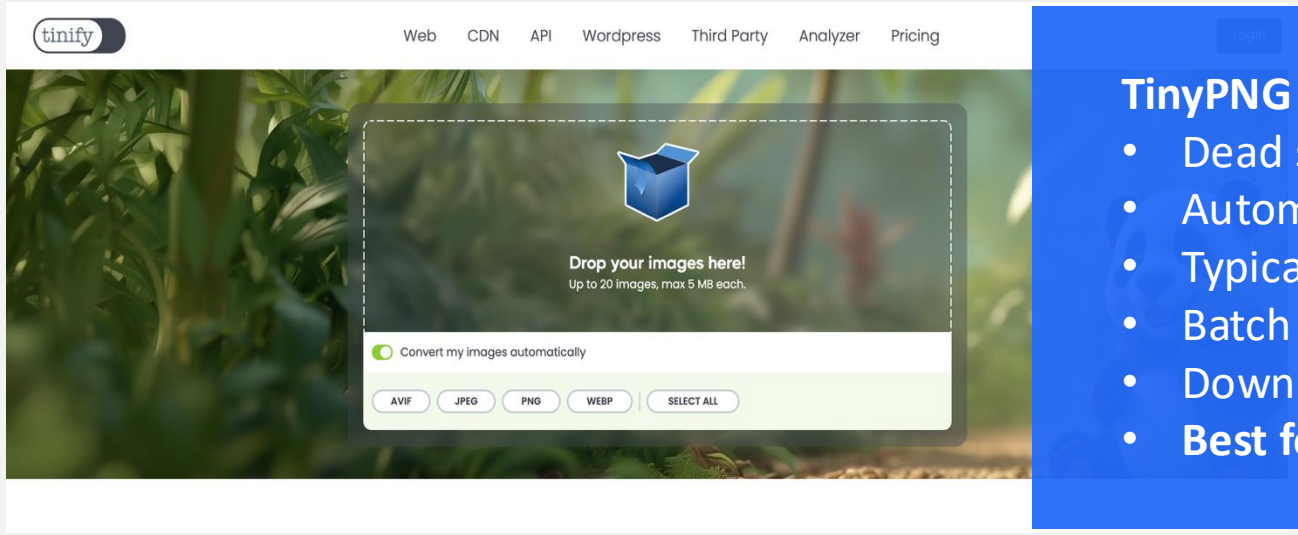
The Sweet Spot: 70-85% quality

- Visually indistinguishable from original
- 50-70% smaller file size
- Perfect balance



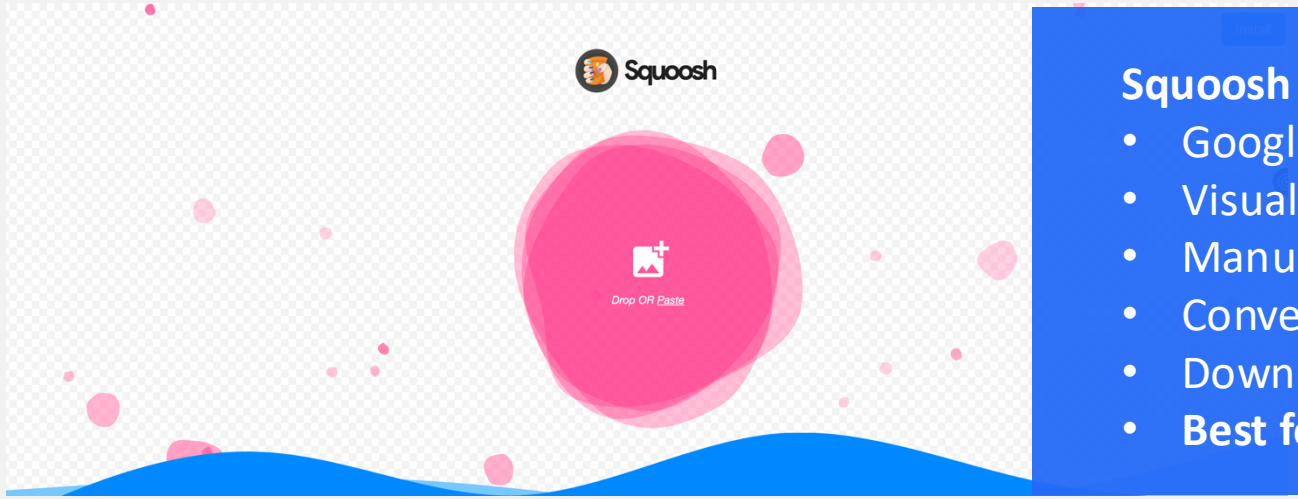
Compression Tools Demo

Free Tools for Easy Image Compression



TinyPNG (tinypng.com)

- Dead simple: Drag and drop images
- Automatic compression (smart defaults)
- Typically 50-70% file size reduction
- Batch process up to 20 images at once
- Download compressed versions
- **Best for:** Quick compression of multiple images

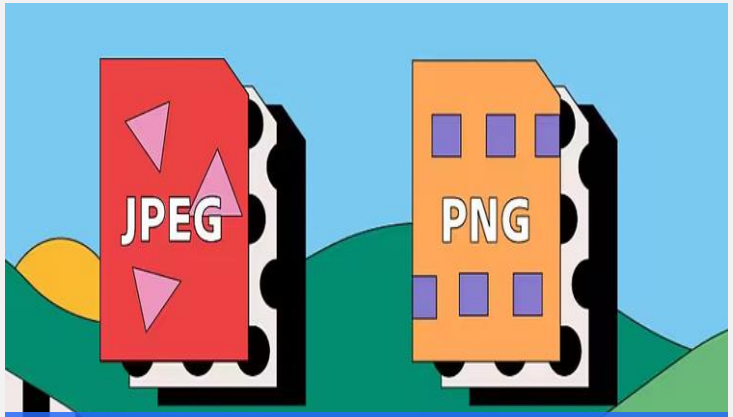


Squoosh (squoosh.app)

- Google's free tool
- Visual before/after slider comparison
- Manually adjust quality with live preview
- Convert between formats (JPG to WebP)
- Download settings for later reuse
- **Best for:** Fine-tuning individual images

Modern Image Formats

New Formats = Smaller Files, Same Quality



JPG/PNG (Traditional Formats)

- The old standard since 1990s
- 100% browser support
- Larger file sizes
- Still widely used



WebP (Modern Format)

- 25-35% smaller than JPG
- Supports transparency (like PNG)
- 97% browser support (2025)
- **Use this for Project 2!**



AVIF (Newest Format)

- 50%+ smaller than JPG
- Excellent quality at tiny sizes
- 92% browser support (2025)
- The future of web images

The Picture Element for Fallbacks

Provide Multiple Formats with Automatic Fallback

```
<picture>
  <source srcset="hero.avif" type="image/avif">
  <source srcset="hero.webp" type="image/webp">
  <img src="hero.jpg" alt="Hero image description">
</picture>
```

How the Browser Chooses:

1. Can I display AVIF? Yes → Use hero.avif (smallest)
2. Can I display WebP? Yes → Use hero.webp (medium)
3. Otherwise → Use hero.jpg (largest, but works everywhere)

Best of Both Worlds:

- Modern browsers get tiny modern formats
- Older browsers get reliable JPG fallback
- You write the code once
- Browser does the work automatically

The browser handles it all automatically

Responsive Images with srcset

Serve Different Image Sizes to Different Devices

```
<img
  srcset="photo-small.jpg 400w,
          photo-medium.jpg 800w,
          photo-large.jpg 1200w"
  sizes="(max-width: 600px) 400px,
         (max-width: 1000px) 800px,
         1200px"
  src="photo-medium.jpg"
  alt="Responsive photo description">
```

What This Does:

- **Mobile (under 600px):** Downloads photo-small.jpg (400px wide)
- **Tablet (600-1000px):** Downloads photo-medium.jpg (800px wide)
- **Desktop (over 1000px):** Downloads photo-large.jpg (1200px wide)

Result: Each device gets exactly the size it needs, nothing wasted.

Responsive CSS Background Images

Serving Different Backgrounds for Different Screens

Why This Matters: Mobile users download only the small image, not the huge desktop version.

```css
/* Desktop - large, high-quality image */
.hero {
  background-image: url('hero-large.jpg');
  background-size: cover;
  background-position: center;
}

/* Tablet - medium image */
@media (max-width: 1024px) {
  .hero {
    background-image: url('hero-medium.jpg');
  }
}

/* Mobile - small, optimized image */
@media (max-width: 600px) {
  .hero {
    background-image: url('hero-small.jpg');
  }
}
```

Lazy Loading Explained

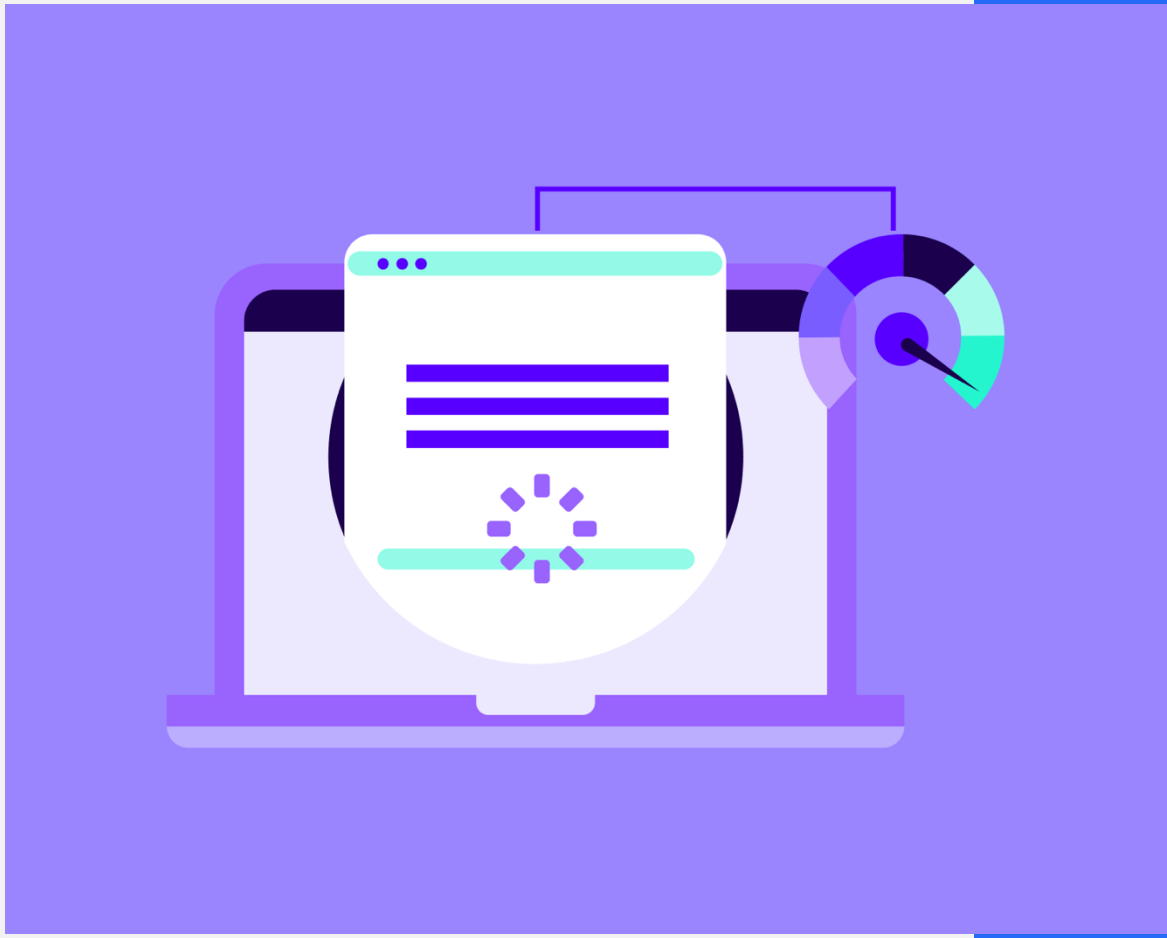
Content: What is Lazy Loading?

Without Lazy Loading:

- Browser loads ALL images immediately when page opens
- User waits for entire page (even unseen images)
- Wastes bandwidth on images user may never see
- Slower initial page load

With Lazy Loading:

- Only visible images load immediately
- Below-fold images wait until user scrolls
- Page appears much faster
- Saves data on images never viewed
- Better user experience



Load images only when user actually needs to see them.

Native Lazy Loading

Built Into Browsers - Incredibly Easy!

```
<!-- Images below the fold - lazy load them -->
<img src="article-photo.jpg"
     loading="lazy"
     alt="Article photo description">

<!-- Hero image above fold - load immediately -->
<img src="hero.jpg"
     loading="eager"
     alt="Hero image description">

<!-- Default behavior if you don't specify -->
<img src="regular.jpg"
     alt="Loads normally">
```

That's it! One attribute. Browser handles everything else.

- **Browser Support:** 97%+ (Chrome, Firefox, Safari, Edge)
- **No JavaScript required. No external libraries. Just one HTML attribute.**

When to Lazy Load?

DO Lazy Load:

- Images below the fold (not initially visible)
- Long pages with many images
- Gallery or portfolio pages with dozens of photos
- Article images throughout body content
- Footer images
- Secondary content images

DON'T Lazy Load:

- Hero images (above the fold)
- Logo in header
- Navigation icons or buttons
- Any critical UI elements
- First screen content
- Images needed for initial user interaction



The Rule: First screen content = eager loading. Everything else = lazy loading.

Resource Hints – Help the Browser

Give the Browser Advance Information

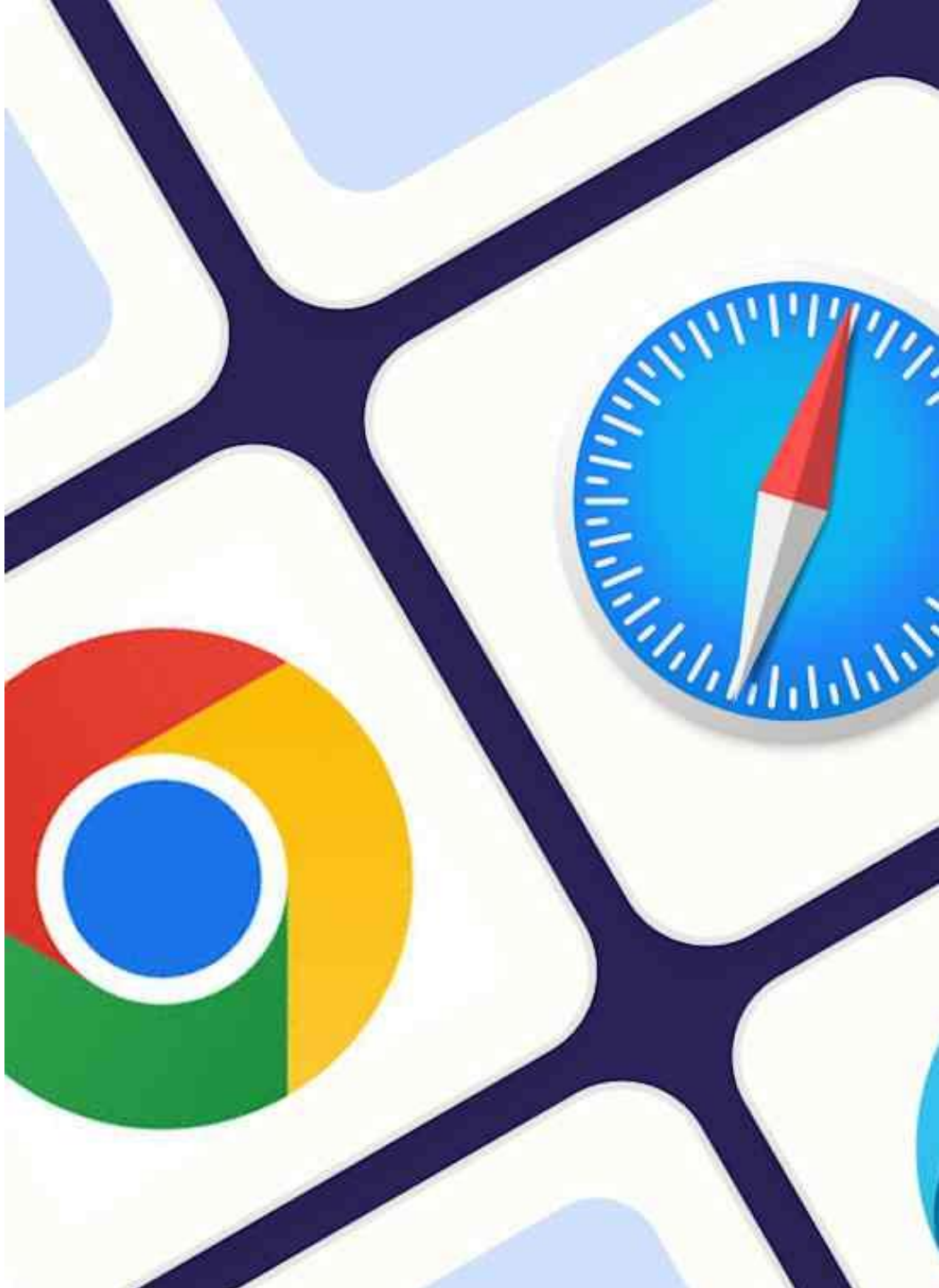
What are Resource Hints?

Special HTML tags that tell the browser what resources are coming so it can prepare ahead of time.

Four Types of Hints:

- **Preload** - "Download this NOW, we'll need it immediately”
- **Prefetch** - "Download this when idle, we'll need it later”
- **Preconnect** - "Establish connection to this server now" **DNS**
- **Prefetch** - "Look up this domain name in advance"

The Goal: Reduce waiting time by preparing resources before they're actually needed.



Preload - High Priority Resources

Preload: "Download This Immediately!"

What Preload Does:

- Tells browser: "This resource is critical, download it ASAP"
- Increases download priority
- Starts downloading before browser naturally discovers it
- Reduces wait time for critical resources

Use Preload For:

- Critical above-fold images (hero images)
- Fonts you use in visible content
- Essential CSS files
- Resources needed immediately on page load

```
<!-- Critical hero image -->
<link rel="preload" href="hero.jpg" as="image">

<!-- Main stylesheet -->
<link rel="preload" href="main.css" as="style">

<!-- Font you definitely use -->
<link rel="preload" href="font.woff2" as="font" crossorigin>
```

Warning: Only preload truly critical resources. Don't overuse it!

Prefetch - Low Priority Resources

Prefetch: "Download When You Have Time"

What Prefetch Does:

- Tells browser: "Get this when you're not busy"
- Low priority, background download
- Ready instantly when user navigates to next page
- Downloads during browser idle time

Use Prefetch For:

- Next page in a sequence
- Resources for likely next user action
- Images for upcoming content
- Predictable navigation paths

```
<!-- Next page user will likely visit -->
<link rel="prefetch" href="about.html">

<!-- Image for next article -->
<link rel="prefetch" href="next-article-image.jpg">

<!-- Resources for likely next step -->
<link rel="prefetch" href="project-details.css">
```

Perfect for multi-page sites where you know where users go next!

Preconnect and DNS-Prefetch

Speed Up External Resource Connections

Preconnect: Full Early Connection

```
<!-- Connect to Google Fonts server early -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

<!-- Connect to CDN early -->
<link rel="preconnect" href="https://cdn.example.com">
```

DNS-Prefetch: Just Resolve Domain Name

```
<!-- Lighter alternative - just DNS lookup -->
<link rel="dns-prefetch" href="https://fonts.googleapis.com">
```

Use For:

- External resources (Google Fonts, CDNs)
- Third-party services
- External image sources

CSS Optimization - Write Efficient Selectors

Minimize Selector Specificity

Inefficient Approach:

```css
/* Overly specific - browser works harder */
body div.container ul.nav li a {
  color: blue;
}
```

- Browser has to check 6 levels deep
- Slower to match elements
- Harder to override later
- Unnecessary complexity

Efficient Approach:

```css
/* Simple class - browser works less */
.nav-link {
  color: blue;
}
```

- Browser checks one class
- Fast to match
- Easy to maintain
- Clear and readable

Principle: Simpler selectors = Faster rendering

CSS Best Practices

Write Less CSS, Load Faster

Combine Selectors:

```css
/* Repetitive - wastes bytes */
h1 { font-family: Arial; }
h2 { font-family: Arial; }
h3 { font-family: Arial; }

/* Combined - efficient */
h1, h2, h3 { font-family: Arial; }
```

Use Shorthand Properties:

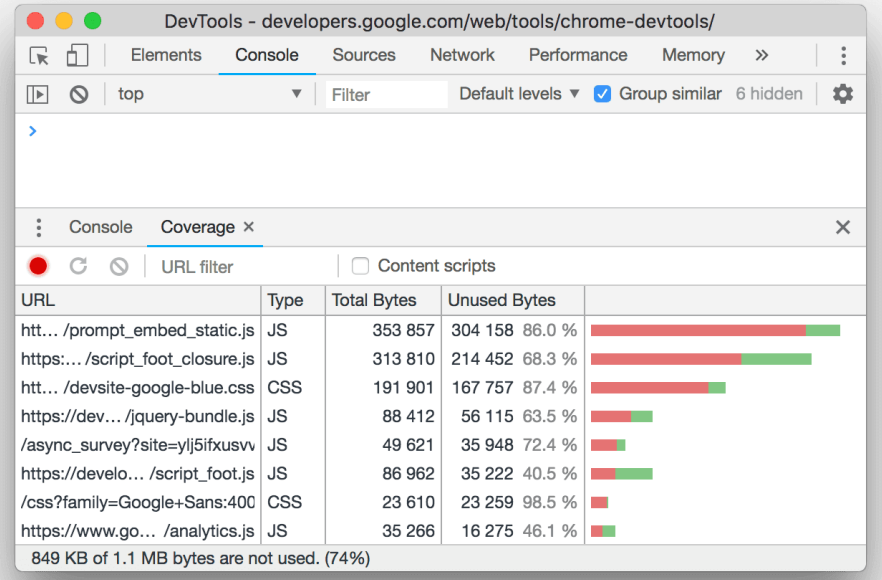
```css
/* Long way - 4 lines */
header {
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 10px;
    margin-left: 20px;
}

/* Shorthand - 1 line */
header {
    margin: 10px 20px;
}
```

Result: Smaller file size, faster download, easier to maintain

Remove Unused CSS

Delete What You Don't Use



The Problem:

- You copy CSS from tutorials or frameworks
- You experiment with styles then abandon them
- You comment out code "just in case"
- Result: Unused CSS bloating your file

Find Unused CSS: Chrome DevTools → Coverage Tab

- Shows unused CSS in red
- Shows used CSS in green
- Helps you identify what to delete

For Project 2:

- Keep your CSS organized by page/section
- Only include styles you actually use
- Delete experimental or "maybe later" styles
- Comment sparingly - comments add file size too

Less CSS = Faster Loading

CSS Minification (Production)

Compress CSS for Production

Before (Development - Readable):

```css
.button {
  background-color: #3498db;
  padding: 10px 20px;
  border-radius: 5px;
  font-size: 16px;
}
```

What Gets Removed:

- Spaces and indentation
- Line breaks
- Comments
- Unnecessary characters

After (Production - Minified):

```css
.button{background-color:#3498db;padding:10px 20px;border-radius:5px;font-size:16px}
```

For Projects: Optional but good to know

HTML Optimization - Keep It Flat

Minimize DOM Depth

Inefficient - Too Deep:

```
<!-- 5 levels deep for one paragraph -->
<div>
  <div>
    <div>
      <div>
        <p>Text content</p>
      </div>
    </div>
  </div>
</div>
```

- Browser traverses 5 levels
- Slower rendering
- Harder to style and maintain

Efficient - Flat Structure:

```
<!-- Simple and direct -->
<div class="container">
  <p>Text content</p>
</div>
```

- Browser traverses 2 levels
- Faster rendering
- Easier to work with

Principle: Fewer HTML elements = Faster page rendering

Project 2 Performance Goals

Your Performance Requirements for Project 2

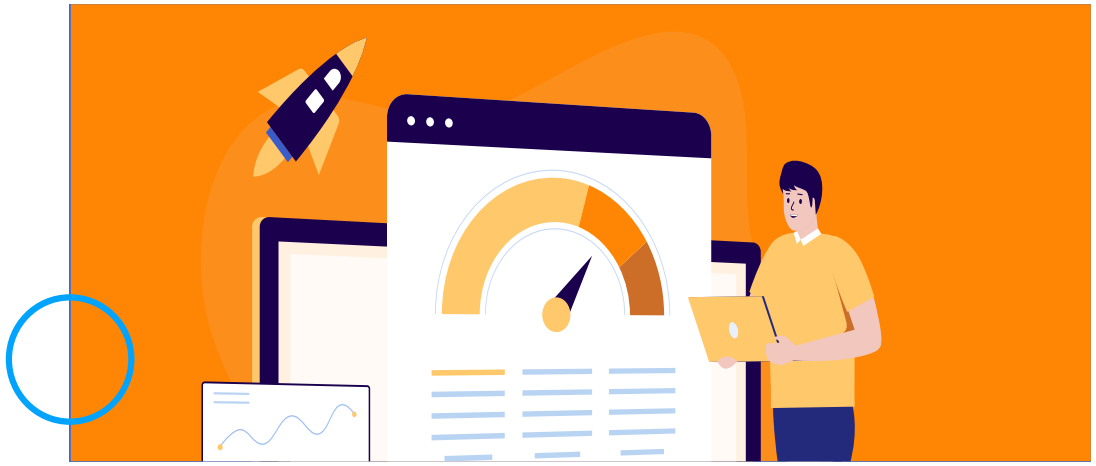


Image Requirements:

- All images optimized and compressed
- No single image over 200 KB
- Use WebP format (with JPG fallback)
- Lazy loading on all below-fold images
- Responsive images using srcset where appropriate

Page Performance:

- Total page size under 2 MB per page
- Page loads in under 3 seconds on Fast 3G
- Lighthouse Performance score: 80 or higher
- No render-blocking resources in head

Code Quality:

- Clean, minimal CSS (no excessive unused styles)
- Semantic HTML structure
- Scripts deferred or at end of body
- Google Fonts preconnected if used

Test as You Build

Test Performance Frequently, Not Just at the End

After Every Major Change:

- Added several images? Test now.
- Created a new page? Test now.
- Implemented new layout? Test now.
- Added external resources? Test now.

Quick Testing Workflow:

1. Open Chrome DevTools (F12)
2. Go to Network tab
3. Refresh page and check total size
4. Throttle to Fast 3G and test load time
5. Run Lighthouse (Performance audit)
6. Fix issues immediately

Why Test Early and Often:

- Catch problems when they're easy to fix
- Avoid huge refactoring at the end
- Build good habits
- Stay within performance budget



Don't wait until the night before Project 2 is due!

Resources and Tools

Bookmark These Tools - You'll Use Them Constantly

Image Compression:

TinyPNG (tinypng.com)

- Simple drag-and-drop compression
- Batch process up to 20 images
- Free, no account needed

Squoosh (squoosh.app)

- Google's compression tool
- Visual quality comparison slider
- Format conversion (JPG to WebP)

ImageOptim (imageoptim.com)

- Mac-only desktop app
- Batch optimization
- Drag folder, optimize everything

Format Conversion:

Squoosh (squoosh.app)

- Convert to WebP or AVIF
- Adjust quality settings

CloudConvert (cloudconvert.com)

- Batch format conversion
- Supports many formats

Performance Testing:

Chrome DevTools (Built into Chrome)

- Network tab, Lighthouse, Coverage

PageSpeed Insights (pagespeed.web.dev)

- Google's public testing tool
- Test any URL


Image optimization?
Project 2?
HAVE ANY
QUESTIONS?
Lazy loading?
CSS and HTML optimization?
Resource hints?
Anything unclear?